

Lab 04

Struct and Function

1 Review

1.1 Function

- Functions are reusable blocks of code that perform specific tasks.
- They enhance code organization, modularity, and readability.
- Functions can take input parameters, process data, and return values.
- By breaking down complex problems into smaller, manageable functions, programmers can improve code efficiency and maintainability.
- C++ provides both built-in library functions and the ability to define your functions.

1.2 Structure

- A structure is a user-defined data type that groups variables of different types under a single name.
- You can define members (variables) within a structure to represent related data. It's like a blueprint for creating custom data structures.
- Structures are useful for organizing complex data, such as representing a person with name, age, and address.
- By using structures, you can improve code readability and maintainability.

2 Requirements

2.1. In-class assignment

2.1.1. P02 (in 'W03.b-Loop Statement-Problems.pdf')

$$S(n) = 1^2 + 2^2 + \dots + n^2$$

Input Data:

A non-negative integer n . $0 \leq n \leq 10^9$.

Output Data:

Sum

Example:

Input	Output
5	55

2.1.2. P20 (in 'W03.b-Loop Statement-Problems.pdf')

List all divisors of the positive integer n .

Input Data:

A positive integer n . $0 < n \leq 10^9$.

Output Data:

The divisors. Each number is separated by a space.

Example:

Input	Output
6	1 2 3 6

2.1.3. P23 (in 'W03.b-Loop Statement-Problems.pdf')

Count the number of "divisors" of the positive integer n .

Input Data:

A positive integer n . $0 < n \leq 10^9$.

Output Data:

The number of divisors.

Example:

Input	Output
6	4

3.1.4. P01 (in “W04-Problems-Structure and File handling.docx”)

Define a struct Date (day, month, year) and the following functions.

1. Input a date (from text file).
2. Output a date (yyyy-MM-dd) (to text file).
3. Output a date with a given format (to text file). For example: yy: 70, yyyy: 1970, M: 8 or 12, MM: 08 or 12, MMM: Jan ... Dec, d: 1 or 29, dd: 01 or 29.

```
outputDateWithFormat(d1, "yyyy-MM-dd"); // 2019 - 12 - 04
outputDateWithFormat(d1, "MMM dd yy"); // Dec 12 19
```

4. Compare 2 date.
5. Find tomorrow: increase a date by 1 day.
6. Find yesterday: decrease a date by 1 day.
7. Increase a date by k day.
8. Decrease a date by k day.
9. Compute the distance between input date vs 1/1/same year.
10. Compute the distance between input date vs 1/1/1970.
11. Compute the distance between 2 input dates.
12. Given that the date of week of 1970-01-01 is Thursday. Compute the date of week of the input date.

2.2. Homework

2.2.1. P02 (in “W04-Problems-Structure and File handling.docx”)

Define a struct Student (id, full name, gpa, address, date of birth) and the following functions.

15. Input a student (from text file).
16. Output a student: id, full name, gpa, address, dob (to text file).
17. Output a student: id, first name (first word), last name (remaining words), gpa, address, dob, age (18 year 2 months, 2 days) (to text file).
18. Extract the class of input student (to text file). For example, if your id is 19127001, you are in K19 class.
19. Compare 2 students by id.
20. Compare 2 students by gpa, and then by id.

21. Compare 2 students by name, and then by id.
22. Compare 2 students by first name, and then by id.
23. Compare 2 students by last name, and then by id.
24. Compare 2 students by dob, and then by id.

2.2.2. P03 (in “W04-Problems-Structure and File handling.docx”)

Define a struct Course (course id, e.g. CSC10001, course name, e.g. Introduction to Programming, and an array of students, status, e.g. open, or close, maximum number of students in the course, minimum number of students in the course) and the following functions.

27. Input a course (from text file).
28. Output a course: id, name, students' info (to text file)
29. Add a student to a course. Check the maximum number of students in the course condition.
30. Remove a student to a course. Check minimum number of students in the course to update the course status: open or close.
31. Get the current date in your computer. Find all students who were born in this month and print the result.
32. Get the current date in your computer. Find all students who were born in this date and print the result.
33. Get the current date in your computer. Find all students who are legal to have driving licences (≥ 18 years old) print the result.
34. Find all students are in K19 class and print the result (to text file).
35. Find a student by a given id and print the result (to text file).
36. Find a student by a given name (“thanh” vs “Tuan-Thanh Ho”). Then print the result (to text file).
37. Sort students by id. Then print the result (to text file).
38. Sort students by first name. Then print the result (to text file).
39. Sort students by gpa. Then print the result (to text file).
40. Sort students by dob. Then print the result (to text file).

3 Guidelines

Write at least 1 function for each problem.

Write a main() function, which a menu that allows user to choose a problem to be executed. If that user chooses 0, exit the program.

```
// <YOUR STUDENT ID>
// <YOUR FULL NAME>
// <YOUR CLASS>

// Test case 1
// Input:
// Output:
```

```
...  
// Test case 3  
// Input:  
// Output:  
  
int main(){  
    //your code here  
  
    return 0;  
}
```